UNIT - 2

Divide-and-Conquer Method: The general method, Binary search, Finding maximum and minimum, Merge sort, Quick sort.

Brute Force: Knapsack, Traveling salesman problem, Convex-Hull

==========

Divide-and-Conquer Method

Divide: Divide the big problem into some sub problems.

Conquer: Calling the sub problems recursively until we get sub problems solutions.

Combine: Combine the sub problem solutions, So that we will get original problem solutions.

Applications of Divide and Conquer:

- → Finding maximum and minimum elements
- → Power of an element.
- → Binary Search.
- → Merge Sort.
- → Quick Sort.
- → Selection procedure.
- → Counting # inversions.
- → Finding maximum contiguous subarray sum.
- → Strassen's matrix multiplications.

DAC - General Method

```
DAC (a, p, q)
{
        if (small (a, p, q))
        return (solution (a, p, q))
        else
        {
                Mid = Divide (a, p, q)
                x= DAC (a, p, q)
                y= DAC (a, mid+1, q)
                z = combine (x, y)
                return (z)
        }
```

}

Recurrence Relation for the above General method

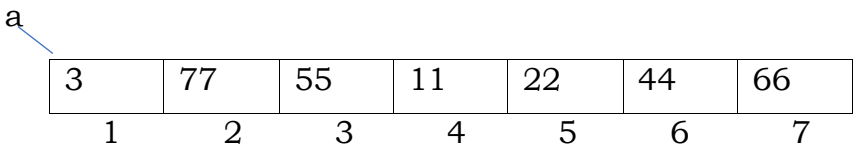
$$T(n) \begin{cases} O(1) & \text{if n is small} \\ f1(n) + T(n/2) + T(n/2) + f2(n) & \text{if n is big} \end{cases}$$

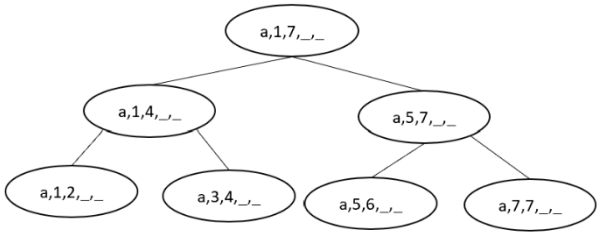
$$T(n) = 2T(n/2) + f(n)$$

Finding maximum and minimum elements in an element.

input: An array of n-distinct integers.

output: Find maximum and minimum elements.

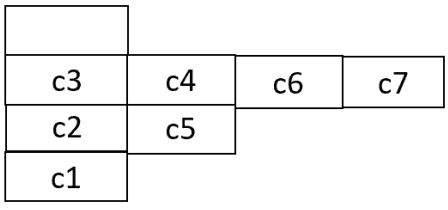




Function call order: C1, C2, C3, C4, C5, C6, C7--> Preorder

Execution order: C3, C4, C2, C6, C7, C5, C1--> Postorder.

Stack Representation



Min - Max algorithm

- Min-Max algorithm is a decision-making algorithm.
- It helps determines the optimal solution.

Steps of the Algorithm

1. Start from the current state (root node).
2. Explore all possible moves (child nodes).
3. Recursively compute the score of each move.
4. At **MAX nodes**, choose the move with the highest score.
5. At **MIN nodes**, choose the move with the lowest score.
6. Return the best score to the top level.

Algorithm for Max_Min

```
DACMax_Min (a, i, j)
{
        if (i==j) //1 element No comparision
        max=min=a[i];
        return (max, min);
        if(i==j-1)  //2 elements 1 comparision
        {
                If(a[i]>a[j])                                   TC = O(1)
                max=a[i], min=a[j];
        else
        {
                max=a[j], min=a[i];
                return (max,min)
        }
        else{
                mid=(I + j)/2; ---------------------→      C time
        (max1, min) = DACMax_Min (a, i, mid);       →T(n/2)
        (max2, min2) = DACMax_Min (a, mid +1, j);          →T(n/2)
                If(max1<max2)
                        max=max2;
                        else
                        max=max1;
                if(min1>min2)                               C time
                        min=min2
```

```
                else
                min=min1;
            return (max, min);
        }
}
```

Recurrence Relation:

$$T(n)= \begin{cases} O(1) & \text{if } n<=2 \\ C+2T(n/2) + C & \text{if } n>2 \end{cases}$$

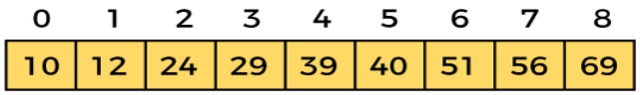
Binary Search Algorithm

- Binary Search approach follows "divide and conquer."
- It divides the list into two parts and compares the target item with the Element.
- Remember that Binary Search is effective on lists.
- If your list is unsorted, sorting it beforehand is essential before employing this method

Working of Binary search – Recursive approach

The recursive method of binary search follows the divide and conquer approach.

Let the elements of array are –



Let the element to search is, K = 56

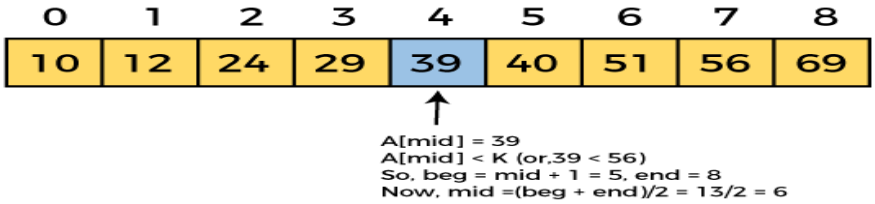
Calculate the **mid** of the array → mid = (beg + end)/2

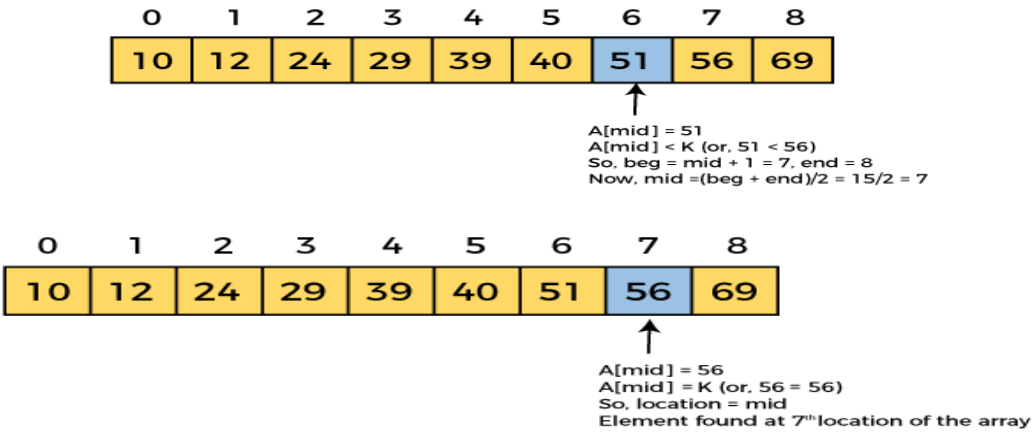
So, in the given array -

beg = 0

end = 8

mid = (0 + 8)/2 = 4. So, 4 is the mid of the array.





Now, the element is found. So, algorithm will return the index of the element matched.

Algorithm - BinarySearch

```
BinarySearch (array, target):
    low ← 0
    high ← length(array) - 1

    while low ≤ high:
        mid ← (low + high) // 2

        if array[mid] == target:
            return mid  // Target found
        else if array[mid] < target:
            low ← mid + 1
        else:
            high ← mid - 1

    return -1  // Target not found
```

Binary Search Time complexity

Case	Time Complexity
Best Case	O (1)
Average case	O (log n)
Worst Case	O (log n)

MergeSort

→ MergeSort algorithm is an application of Divide and Conquer

→ input: Sorted/unsorted elements of an array

→ output: Sorted array

→ Merge ( ) is an important function

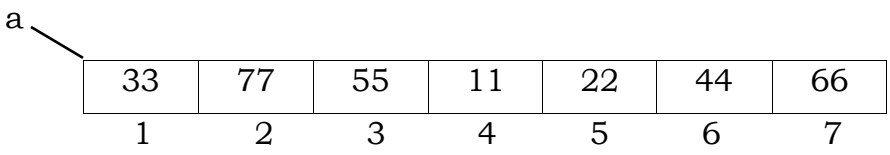
→ MergeSort is a Super set of Merge

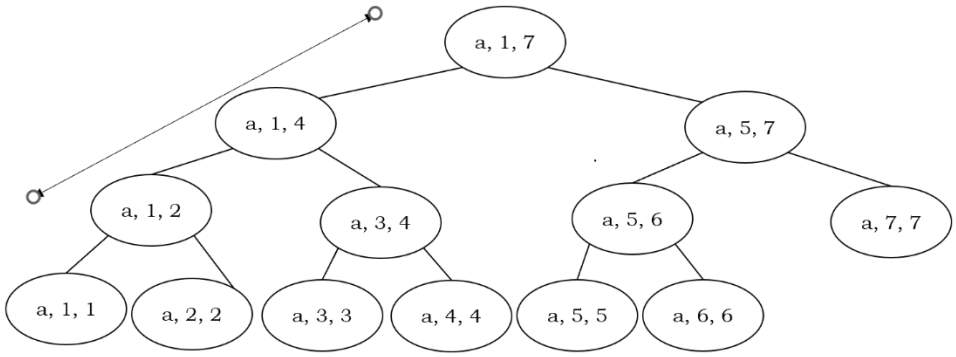
→ Merge sort divides the elements Blindly

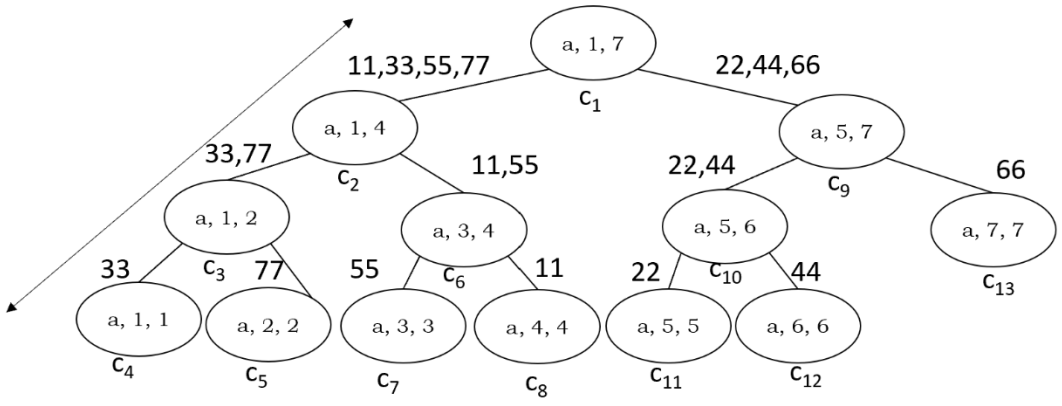
→ Merge ( ) combines the elements meaningfully

→ MergeSort is an outplace algorithm

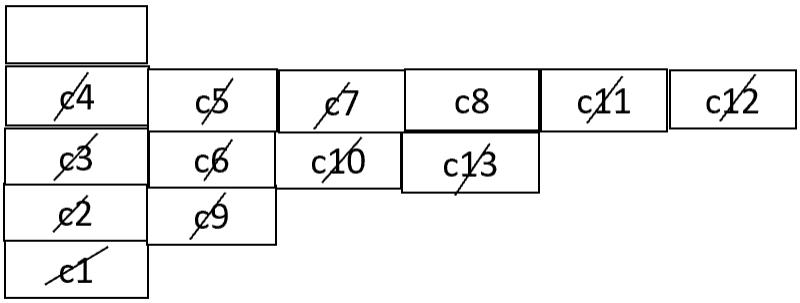
→ MergeSort is a Stable algorithm.

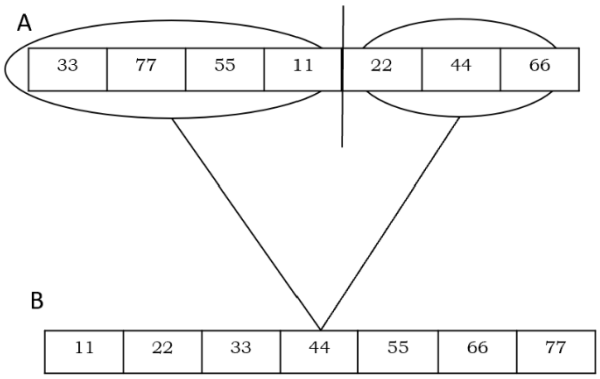






Stack Representation





Merge Algorithm (for two sorted arrays)

1. Initialize two pointers, one for each array.
2. Compare the elements at the pointers.
3. Insert the smaller element into a new result array.
4. Move the pointer of the array from which the element was taken.
5. Repeat until all elements from both arrays are merged.

```
merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
```

```
            j += 1

        # Add any remaining elements
        result.extend(left[i:])
        result.extend(right[j:])
            return result
```

Merge Sort Pseudocode (Recursive Version)

```
merge_sort(arr):
    if len(arr) <= 1:
        return arr

    # Step 1: Divide
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Step 2: Merge
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    # Compare elements and merge
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Append any remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Final Time Complexity

- Best case: O(n log n)
- Average case: O(n log n)
- Worst case: O(n log n)

Space Complexity

O(n) for the auxiliary array used in merging.

Quick Sort Algorithm

→ Quick Sort is sort is an application of Divide and Conquer.

→ i/p: for Quick sort is an Unsorted array.

→ o/p: is an Sorted array.

→ Quick sort is not stable algorithm Order of elements may or may not be maintained.

→ Quick is an in-place algorithm, It won't need any extra space.

→ Partition algorithm plays an important role in Quick sort.

→ Partition algorithm divides the array meaningfully.

a

50	80	90	15	100	35	150	11
1	2	3	4	5	6	7	8

Algorithm for Partition

```
partition (a, p, q)
{
        x=a[p];
        i=p;
        for(j=p+1; j<=q: j++)
        {
                if(a[j] < x)
                {
                        i++:
                        swap(a[i],a[j])
                }
                swap(a[p], a[i])
                return (i);
        }
}
```

Algorithm for Quick Sort

```
QuickSort(a, p, q)
{
        if(p == q)
        return a[p];
        else
        {
                mid = partition(a,p,q);
                QuickSort(a,p,mid-1);
                QuickSort(a,mid+1,q);
                return (a);
        }
}
```

Recurrence Relation:

$$T(n)= \begin{cases} O(1) & \text{if } n=1 \\ n+2T(n/2) & \text{if } n>1 \end{cases}$$

Time Complexity for Quick Sort is

For partition Algorithm needs O(n).

Quick Sort Algorithm needs log n levels to sort.

So, T.C is O (n log n).

Brute Force Algorithm

A brute force algorithm tries all possible solutions until it finds the correct one. It does not use any shortcuts — it just checks every option.

Steps of a Brute Force Algorithm

1. Understand the Problem
   - Knows the problem try to solve.
     Example: Find a number in a list or solve a password.
2. List All Possible Solutions
   - Generate every possible option.
   - No guess — just try every option.
3. Test Each Solution
   - Go through each possible solution one by one.

- Check if it solves the problem.

4. Return the Correct Solution
   - As soon as you find the right answer, stop (or continue checking all if multiple answers are needed).
5. (Optional) If No Solution Found
   - If no solution found or return failure.

Example: Search for a number in a list
list: [5, 8, 2, 9, 4] and to find 9.
Brute force steps:

1. Start with the first number: 5 → not 9
2. Check the second number: 8 → not 9
3. Check the third number: 2 → not 9
4. Check the fourth number: 9 → found it!
5. Return the position (index 3).

Example: Cracking a Password
Suppose a password is a 2-letter combination like "ab", "ac", etc.
Brute force steps:

1. Try "aa" → fail
2. Try "ab" → fail
3. Try "ac" → fail
4. …
5. Keep going until you find the correct password.

Pros and Cons of Brute Force

Pros	Cons
Easy to understand and implement	Very slow for big problems
Finds a solution if one exists	Not smart, wastes time and resources

Knapsack Problem

- A knapsack with a maximum weight it can hold (say, W).
- A list of items, each with:
  - A weight and
  - A value (profit).

Goal: Select a set of items to maximize the total value without exceeding the weight capacity.

Solving Knapsack using Brute Force

In brute force,

- Try all possible combinations of items.
- Check if each combination fits in the knapsack (weight ≤ W).
- Keep track of the combination with the highest value.

Step 1: List all possible subsets of items

- Each item can either be:
  - Included in the knapsack, or
  - Not included.
- For n items, there are 2^n subsets (combinations).

Example: If you have 3 items, the subsets are:
{}, {Item1}, {Item2}, {Item3}, {Item1, Item2}, {Item1, Item3}, {Item2, Item3}, {Item1, Item2, Item3}

Step 2: Calculate total weight and value for each subset

- For each subset:
  - Add up the weights.
  - Add up the values.

Step 3: Check if the subset is valid

- If the total weight ≤ W (capacity), it's a valid subset.
- Otherwise, ignore it.

Step 4: Find the subset with the maximum value

- Among all valid subsets, pick the one with the highest total value.

Example

Items	I_1	I_2	I_3	I_4
Weight	7	3	4	5
Value	42	12	40	25

Solution

Subsets	Total Weight	Total Value
I_1	7	42
I_2	3	12
I_3	4	40
I_4	5	25
I_1, I_2	10	54
I_1, I_3	11 X	
I_1, I_4	12 X	
I_2, I_3	7	52
I_2, I_4	8	37
I_3, I_4	9	**65**
I_1, I_2, I_3	14 X	
I_1, I_2, I_4	15 X	
I_1, I_3, I_4	16 X	
I_2, I_3, I_4	12 X	
I_1, I_2, I_3, I_4	19 X	

```
function KnapsackBruteForce (weights, values, n, W):
    return KnapsackHelper (weights, values, n, W, 0, 0, 0)

function KnapsackHelper (weights, values, n, W, i, currentWeight, currentValue):
    if i == n:
        if currentWeight <= W:
            return currentValue
        else:
            return 0

    // Case 1: Include the item if it doesn't exceed capacity
    include = 0
    if currentWeight + weights[i] <= W:
        include = KnapsackHelper(weights, values, n, W, i + 1,
                                 currentWeight + weights[i],
                                 currentValue + values[i])
```

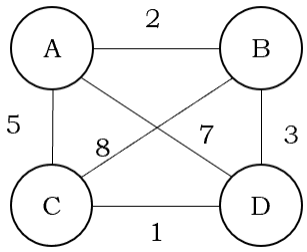
```
// Case 2: Exclude the item
  exclude = KnapsackHelper (weights, values, n, W, i + 1,
                            currentWeight,
                            currentValue)

  return max (include, exclude)
```

Time Complexity: O(2^n)
Due to recursive call stack: **O(n)** (maximum depth = n)

Travelling Salesman Problem

The Travelling Salesman Problem (TSP) is an optimization problem. It involves finding the shortest possible route that visits each city exactly once and returns to the starting city.

This problem can also be considering as Hamiltonian circuits.
Example:



Step 1: Start from any node exactly visit each node once and get back to where it has started.
Step 2: compute all the possible paths from that node and calculate the distance between the nodes.
Step 3: Then find out the optimal solution.

Working process

Consider the starting vertex is A.

Find all the outgoing edges from A, B, C and D

A → B → C → D
A → C → B → D
A→ D → B → C
A → B → D → C
A → C → D → B
A → D → C → B

According to Hamiltonian Circuit, go to starting node → that are feasible solutions.

A → B → C → D → A
A → C → B → D → A
A→ D → B → C → A
A → B → D → C → A
A → C → D → B → A
A → D → C → B → A

All the above are optimal solutions. But we need an optimal solution. To get optimal solution, compute path distance

A → B → C → D → A = 2 + 8 + 1 + 7 = 18
A → C → B → D → A = 5 + 8 + 3 + 7 = 23
A→ D → B → C → A = 7 + 3 + 8 + 5 = 23
A → B → D → C → A = 2 + 3 + 1 + 5 = 11
A → C → D → B → A = 5 + 1 + 3 + 2 = 11
A → D → C → B → A = 7 + 1 + 8 + 2 = 18

Optimal solutions are

A → B → D → C → A = 2 + 3 + 1 + 5 = 11
A → C → D → B → A = 5 + 1 + 3 + 2 = 11 with minimum cost.

Step by Step explanation

- *Start from a city.*
- *Recursively visit all **unvisited cities** one by one.*
- *At each step, calculate the total distance.*
- *Return to the starting city when all cities are visited.*
- *Keep track of the **minimum tour cost**.*

Pseudocode for TSP

```
function TSP(currentCity, visitedCities):
    if all cities are visited:
        return distance from currentCity to startCity

    minCost = ∞

    for each city in cities:
        if city is not in visitedCities:
            cost = distance[currentCity][city] +
                   TSP(city, visitedCities + {city})
            minCost = min (minCost, cost)

    return minCost
```

Time Complexity

In the brute-force divide and conquer approach to the Travelling Salesman Problem (TSP):

From each city, you try all possible remaining cities.

This results in checking all permutations of cities.

For n cities, the number of possible tours is:

Time Complexity = O(n!)

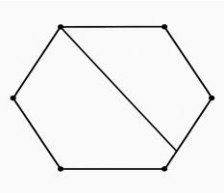
Convex-Hull

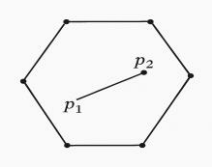
Definition

- The convex hull of a set of points S is the smallest convex polygon that contains all the points in the set S.
- A polygon is convex if every internal angle is less than or equal to 180° and for every pair of points inside the polygon, the line segment joining them is also entirely inside the polygon

Properties of a Convex Hull

1. Convexity: Any line segment joining two points inside the hull lies completely inside the hull.
2. Minimal Area: Among all convex shapes that enclose the points, the convex hull has the minimal area/perimeter.
3. Uniqueness: For a given set of points, the convex hull is unique.

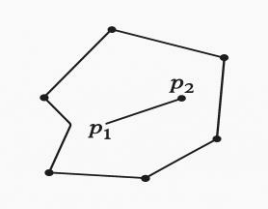




Example

- You have some nails hammered into a board (each nail is a point).
- You stretch a rubber band around the outermost nails.
- When released, the rubber band snaps tightly around the nails forming a polygon.
- That polygon is the convex hull of the set of points.

Not convex hull



The diagram shows a non-convex polygon—a polygon with at least one interior angle greater than 180°. Specifically:

- It has six vertices, connected to form a closed shape.
- One of the edges creates an inward dent (concave angle), clearly indicating it's not a convex polygon.
- Inside the polygon, there is a line segment labeled p_1 and p_2 connecting two interior points.
- The segment p_1 and p_2 lies entirely within the polygon, demonstrating that while this specific segment is inside the polygon, the polygon itself is not convex, and therefore not a convex hull of its vertices.

Problem Statement

Input: A finite set of points in 2D space
Output: The sequence of points or their indices forming the convex polygon that wraps around all points.

Given a set of points P = {p_1, p_2, …, p_n} in the 2D plane, compute a subset H ⊆ P such that:

- H forms the boundary of a convex polygon.
- All points in P lie on or inside this polygon.

Solution:

Step 1: Find the Anchor Point

Find the point with the lowest y-coordinate (if tie, then lowest x). Let's call this P_0. This is guaranteed to be on the convex hull.

Step 2: Sort Points by Polar Angle

Sort the remaining points by the angle they make with P_0 in counterclockwise order.

For each point p, compute:

$\theta = \text{atan2}(p.y - P_0.y, p.x - P_0.x)$

Step 3: Traverse Sorted Points - Use a stack to build the hull:

1. Push P_0 and the first two sorted points.
2. For each point p:
    - While the last two points on the stack and p make a right turn, pop the top.
    - Push p onto the stack.

Step 4: Result

The stack now contains the vertices of the convex hull in counterclockwise order.

Example: P = [(0, 3), (1, 1), (2, 2), (4, 4), (0, 0), (1, 2), (3, 1), (3, 3)]

1. Lowest Y: (0, 0) → anchor
2. Sort by polar angle from (0,0)
3. Build the hull:
    - Start with (0,0), (1,1), (2,2)
    - Check turns:

- Right turn? Pop!
- Left turn? Push!

4. Final hull: [(0,0), (3,1), (4,4), (0,3)]

Applications

- Pattern Recognition – detecting shape boundaries
- Geographic Information Systems (GIS) – boundary of regions
- Collision Detection in video games and physics simulations
- Image Processing – contour extraction
- Robotics – obstacle navigation and mapping.

===000ooo000===